**TRIBHUVAN UNIVERSITY**

INSTITUTE OF ENGINEERING

BE Programme | Year III / Part II

# EMBEDDED SYSTEM (CT 655)

## QUESTION BANK WITH ANSWERS

Chapter 6: Real-Time Operating System (RTOS)

### COURSE SYLLABUS AT A GLANCE

| Unit | Topic | Hours | Marks |
|---|---|---|---|
| 1 | Introduction to Embedded System | 3 | 4 |
| 2 | Hardware Design Issues | 4 | 8 |
| 3 | Software Design Issues | 6 | 8 |
| 4 | Memory | 5 | 8 |
| 5 | Interfacing | 6 | 8 |
| 6 | Real-Time Operating System (RTOS) | 8 | 12 |
| 7 | Control System | 3 | 8 |
| 8 | IC Technology | 3 | 8 |
| 9 | Microcontrollers in Embedded Systems | 3 | 8 |
| 10 | VHDL | 4 | 8 |
| | **Total** | **45** | **80** |

**Chapter 6 Topics (8 Hours | 12 Marks):**

- 6.1 Operating System Basics — Kernel, Types, Functions
- 6.2 Task, Process, and Threads — States, Life Cycle, TCB
- 6.3 Multiprocessing and Multitasking — Context Switching
- 6.4 Task Scheduling — FCFS, SJF, Priority (Preemptive/Non-Preemptive), Round-Robin
- 6.5 Task Synchronization — Semaphore, Mutex, Race Condition, Deadlock
- 6.6 Inter-Task Communication — Message Queue, Mailbox, Event Flags
- 6.7 RTOS vs GPOS — Comparison, Real-Time Requirements
- 6.8 Device Drivers in RTOS

# Chapter 6: Real-Time Operating System (RTOS)

| Q1. How is RTOS different from GPOS? Differentiate between process and thread. | 2081 Chaitra (Regular) \| 2079 Chaitra (Back) \| 2078 Baishakh (Back) [3+3+6] |
|---|---|

**ANSWER**

**RTOS vs GPOS:**

| Feature | RTOS | GPOS (General-Purpose OS) |
|---|---|---|
| Primary Goal | Meet strict real-time deadlines | Maximize throughput and user fairness |
| Scheduling | Priority-based preemptive; deterministic | Round-robin, multi-level feedback queue |
| Interrupt Latency | Bounded, guaranteed (µs–ms) | Unbounded, variable (ms–seconds) |
| Kernel Size | Small: 10 KB–200 KB | Large: 100 MB–GBs |
| Context Switch Time | Deterministic, minimal | Variable, can be long |
| Memory Management | Fixed-size pools (no fragmentation) | Virtual memory, paging, dynamic alloc |
| Response to Events | Immediate, predictable | Best-effort, not guaranteed |
| File System | Optional, simple (FAT) | Full featured (NTFS, ext4, etc.) |
| Examples | FreeRTOS, VxWorks, uC/OS-II, Zephyr | Windows, Linux, macOS, Android |
| Applications | ABS brakes, pacemakers, industrial ctrl | PCs, servers, smartphones |

**Process vs Thread:**

| Aspect | Process | Thread |
|---|---|---|
| Definition | An independent program in execution with its own address space | Lightweight unit of execution within a process; shares process memory |
| Memory | Own separate address space (code, data, heap, stack) | Shares code, data, heap of parent; own stack only |
| Context Switch | Expensive — save/restore full context + MMU TLB flush | Cheaper — shares most context with other threads |
| Communication | IPC: pipes, message queues, shared memory, sockets | Direct via shared variables (needs synchronization) |
| Creation overhead | High (new address space allocation) | Low (only new stack and registers) |

| | | |
|---|---|---|
| Failure isolation | Crash in one process does not affect others | Thread crash can crash entire process |
| Example | Two separate applications (editor + browser) | Multiple tabs in one browser process |

| | |
|---|---|
| **Q2. Define kernel and differentiate between monolithic kernel and microkernel. Explain the major functions of an RT Kernel.** | **2073 Bhadra (Regular) \| 2076 Baisakh (Back) [3+3+6]** |

**ANSWER**

**Kernel:**

The kernel is the core component of an operating system that manages hardware resources (CPU, memory, I/O devices) and provides essential services to user applications. It runs in a privileged processor mode (kernel mode/supervisor mode) with direct hardware access.

| Feature | Monolithic Kernel | Microkernel |
|---|---|---|
| Structure | All OS services (file system, drivers, network, IPC) compiled into one large kernel binary in kernel space | Only core services (IPC, basic memory, task scheduling) in kernel; everything else in user-space servers |
| Performance | Fast — service calls don't cross kernel boundary | Slower — IPC overhead between user-space servers and kernel |
| Stability | A driver or file system bug can crash entire system | User-space server crash does not crash the kernel |
| Code size | Larger (millions of lines) | Smaller, minimal kernel code |
| Security | Larger attack surface | Better isolation; smaller trusted computing base |
| Examples | Linux, BSD Unix | QNX (RTOS), L4, MINIX 3, Mach |
| RTOS use | Linux with PREEMPT_RT patch | QNX Neutrino (pure microkernel RTOS) |

**Major Functions of an RT (Real-Time) Kernel:**

- Task/Thread Management: Create, delete, suspend, and resume tasks. Assign and change priorities. Maintain Task Control Blocks (TCBs) for each task (state, stack pointer, priority, registers).
- Scheduling: Implement priority-based preemptive scheduling — ensure the highest-priority ready task always executes. Handle priority inversion with priority inheritance.
- Inter-Task Communication (ITC): Provide semaphores, mutexes, event flags, message queues, mailboxes for safe data exchange and synchronization between tasks.
- Timer Services: Software timers for periodic task activation, timeouts, and delays. System tick interrupt drives the scheduler.
- Memory Management: Fixed-size memory pool allocation for deterministic timing — no heap fragmentation. Optional dynamic memory management.

- Interrupt Management: Handle hardware interrupts; provide ISR-safe OS calls (give semaphore from ISR, post message from ISR). Manage interrupt nesting and priorities.
- Device Driver Interface: Standardized API for hardware peripheral access — isolates application from hardware specifics.
- Power Management: Control CPU sleep modes and peripheral power gating between task activations to minimize energy consumption.

| **Q3. Draw and explain the process life cycle (task state diagram) in an RTOS. Explain the Task Control Block (TCB).** | **2081 Ashwin (Back) \| 2077 Magh (Back) [4+2+6]** |
|---|---|

**ANSWER**

**Process (Task) Life Cycle — State Diagram:**

```
                  ┌──────────────┐
Create ──────►    │     NEW      │
                  └──────────────┘
                         │
                Admitted │
                         ▼
                  ┌──────────────┐          ┌──────────────┐
◄─Preempted─      │     READY    │ ◄──────── │ BLOCKED /    │
Run (sched)─►     │  (Runnable)  │  Wakes up │  WAITING     │
                  └──────────────┘          └──────────────┘
                         │                          ▲
             Scheduled   │                          │
                         ▼                Waits/Blocks
                  ┌──────────────┐                  │
                  │   RUNNING    │ ─────────────────►│
                  └──────────────┘
                         │
               Finishes  │
                         ▼
                  ┌──────────────┐
                  │ TERMINATED   │
                  └──────────────┘
```

**State Descriptions:**

- NEW: Task has been created (TCB allocated, stack set up) but not yet admitted to the ready queue.
- READY: Task is ready to execute but waiting for CPU (another task is currently running). Sits in the priority-ordered ready queue.
- RUNNING: Task currently executing on CPU. Only one task per CPU core is in RUNNING state.
- BLOCKED/WAITING: Task is waiting for an event — I/O completion, semaphore, mutex, timer expiry, or message. Does not consume CPU time.
- TERMINATED (DEAD): Task has completed (returned from task function) or been deleted. Resources (TCB, stack) are freed.

**State Transitions:**

- NEW → READY: Task admitted to scheduler (osTaskCreate).
- READY → RUNNING: Scheduler selects this task (highest priority ready task).

- RUNNING → READY: Task preempted by higher-priority task, or time slice expired (RR).
- RUNNING → BLOCKED: Task calls blocking API (osSemaphoreAcquire, osDelay, osMessageGet).
- BLOCKED → READY: Awaited event occurs (semaphore given, timer fires, message arrives).
- RUNNING → TERMINATED: Task completes or calls osTaskDelete.

**Task Control Block (TCB):**

The TCB is a data structure maintained by the RTOS kernel for each task. It stores all information needed to save and restore a task's execution context:

```
typedef struct TCB {
  uint32_t  *pxTopOfStack;    // Current stack pointer (saved SP)
  uint32_t   ulTaskNumber;    // Unique task ID
  char       pcTaskName[16];  // Task name (for debugging)
  uint8_t    uxPriority;      // Task priority (0=lowest)
  uint8_t    eState;          // READY / RUNNING / BLOCKED / ...
  uint32_t   xTicksToDelay;   // Delay counter (for osDelay)
  ListItem_t xStateListItem;  // Node in ready/blocked list
  uint32_t  *pxStack;         // Base of task stack
} TCB_t;
```

| **Q4. What are the different situations when context switching is necessary? Explain the detailed steps involved in context switching.** | **2080 Chaitra (Regular) \| 2074 Bhadra (Back) [2+6]** |
|---|---|

**ANSWER**

**Situations Requiring Context Switch:**

- Preemption: A higher-priority task enters the READY state (becomes unblocked or is created). The scheduler preempts the current lower-priority task.
- Time-slice expiry: In Round-Robin scheduling, the current task's time quantum (tick) expires; scheduler switches to next ready task of equal priority.
- Blocking (Waiting): The current task calls a blocking OS API — waiting for semaphore, mutex, I/O, delay (osDelay), or message queue.
- Task Termination: The running task completes execution; OS selects the next highest-priority ready task.
- Voluntary yield: Task explicitly calls osThreadYield() to give up CPU voluntarily.
- Interrupt completion: After an ISR runs and gives a semaphore/message, if the unblocked task has higher priority, a context switch occurs immediately after ISR exit.

**Detailed Steps in Context Switching:**

```
Step 1: TRIGGER (timer tick ISR fires / task calls blocking API)
        ↓
Step 2: SAVE CONTEXT of current task (Task A)
        Push CPU registers R0-R15, PC, SP, Status/CPSR onto Task A's stack
        (ARM Cortex-M: HW auto-saves R0-R3,R12,LR,PC,xPSR on ISR entry)
        ↓
Step 3: UPDATE TCB of Task A
        Task A's TCB → save current SP into TCB.pxTopOfStack
        Task A state: RUNNING → READY (preemption) or BLOCKED (waiting)
        ↓
Step 4: INVOKE SCHEDULER
        Scheduler scans ready queue; selects Task B (highest priority)
        ↓
Step 5: UPDATE TCB of Task B
        Task B state: READY → RUNNING
        ↓
Step 6: RESTORE CONTEXT of Task B
        Load SP from Task B's TCB.pxTopOfStack
        Pop registers R0-R15, PC, CPSR from Task B's stack
        ↓
Step 7: RESUME EXECUTION
        CPU begins executing Task B at its saved PC
```

Context switch time: 2–20 μs in typical RTOS (FreeRTOS on ARM Cortex-M: ~1–5 μs). Must be minimized for real-time performance.

| **Q5. Explain task synchronization. Define semaphore and mutex. What is priority inversion and how is it solved?** | **2075 Bhadra (Back) \| 2072 Magh (Back) \| 2076 Chaitra (Back) [2+3+3+4]** |
|---|---|

**ANSWER**

### Task Synchronization:

In a multitasking RTOS, multiple tasks may share resources (shared memory, peripheral registers, global variables) or depend on each other's results. Task synchronization ensures correct, ordered access to shared resources and coordinates task execution.

Without synchronization, race conditions occur — the result depends on the order of execution, which is unpredictable.

### Race Condition Example:

```
// Shared variable: counter = 5
// Task A: counter++   (reads 5, adds 1, writes 6)
// Task B: counter++   (also reads 5 before Task A writes!)
// Result: counter = 6 instead of 7 — DATA CORRUPTION
```

**Semaphore:**

A semaphore is a non-negative integer counter maintained by the OS kernel used for signaling between tasks or from ISR to task.

- Binary Semaphore (0 or 1): Used for signaling — Task A gives (posts) the semaphore to signal Task B that an event occurred. Task B waits (pends) on it.
- Counting Semaphore (0 to N): Tracks availability of N identical resources (e.g., N slots in a buffer pool).
- Operations: osSemaphoreAcquire() — decrement (blocks if 0). osSemaphoreRelease() — increment (unblocks waiting task).
- Semaphores do NOT have ownership — any task or ISR can give/take.

**Mutex (Mutual Exclusion Semaphore):**

A mutex is a binary semaphore with ownership tracking. Only the task that acquired the mutex can release it. Used to protect critical sections (shared resources).

```
osMutexAcquire(mutex, osWaitForever);  // Lock — blocks if taken
  // Critical Section — only one task at a time
  shared_variable++;
osMutexRelease(mutex);                 // Unlock
```

- Ownership: Only the owning task can release the mutex — prevents accidental unlocking by another task.
- Mutex supports priority inheritance (see below).

**Priority Inversion:**

Priority inversion is a problem where a high-priority task is blocked by a low-priority task that holds a shared mutex — effectively running at the low priority. If a medium-priority task (that doesn't need the mutex) runs, it prevents the low-priority task from releasing the mutex, blocking the high-priority task indefinitely.

```
High Priority Task (H)  → needs Mutex M → BLOCKED (L holds M)
Medium Priority Task(M) → preempts L (M has higher priority than L)
Low Priority Task (L)   → holds Mutex M but can't run (M preempts it)
Result: H waits indefinitely — PRIORITY INVERSION
```

**Solution — Priority Inheritance:**

When a high-priority task blocks on a mutex held by a lower-priority task, the OS temporarily raises the low-priority task's priority to match the blocked high-priority task. This allows the low task to complete its critical section and release the mutex quickly.

- FreeRTOS mutexes implement priority inheritance automatically.
- Famous real-world case: Mars Pathfinder (1997) experienced priority inversion; rebooting the spacecraft cleared the deadlock.

| Q6. Define deadlock and state the Coffman conditions for deadlock. Explain deadlock handling techniques. | 2082 Kartik (Back) \| 2073 Bhadra (Regular) \| 2078 Chaitra (Back) [6+6] |
|---|---|

**ANSWER**

**Deadlock:**

Deadlock is a situation in which a set of processes (tasks) are permanently blocked — each waiting for a resource held by another process in the set — and no process can ever proceed. The system comes to a standstill.

```
Process P1 holds Resource R1, waiting for Resource R2
Process P2 holds Resource R2, waiting for Resource R1
→ Neither can proceed — DEADLOCK

Resource Allocation Graph: P1 → R2 → P2 → R1 → P1 (circular!)
```

**Coffman Conditions (all four must hold simultaneously for deadlock):**

**1. Mutual Exclusion:** At least one resource is held in a non-shareable mode — only one process can use it at a time. (E.g., a printer, a mutex-protected variable.)

**2. Hold and Wait:** A process holds at least one resource while simultaneously waiting to acquire additional resources currently held by other processes.

**3. No Preemption:** A resource cannot be forcibly taken away from the process holding it; the process must voluntarily release it.

**4. Circular Wait:** A circular chain of processes exists: P1 waits for a resource held by P2, P2 waits for a resource held by P3, ..., Pn waits for a resource held by P1.

**Deadlock Handling Techniques:**

**1. Prevention:** Design the system so that at least one Coffman condition can never hold.

- Break Mutual Exclusion: Make resources shareable where possible (read-only files).
- Break Hold and Wait: Require a process to request ALL needed resources at startup (before starting), or release all held resources before requesting new ones.
- Allow Preemption: If a process holding a resource requests another unavailable resource, forcibly take its held resources away and add them to the waiting list.
- Break Circular Wait: Impose a total ordering on all resource types; processes must request resources in strictly increasing order.

**2. Avoidance (Banker's Algorithm):** Before granting a resource, the OS checks whether granting it would leave the system in a 'safe state' (a state from which all processes can still complete). If not safe, the process waits.

**3. Detection and Recovery:** Allow deadlock to occur; periodically run a deadlock detection algorithm (resource allocation graph cycle detection). When detected, recover by:

- Process termination: Kill one or more deadlocked processes (restart later).
- Resource preemption: Forcibly take resources from a process and assign to another.

**4. Ignorance (Ostrich Algorithm):** Pretend deadlock cannot occur. Used in desktop OSes (Windows, Linux) where deadlock is rare and the cost of prevention/detection exceeds the cost of an occasional reboot.

| | |
|---|---|
| **Q7. Explain the following scheduling algorithms with their advantages and disadvantages: FCFS, SJF, Priority-Based, Round-Robin.** | **2080 Ashwin (Back) \| 2079 Jestha (Back) \| 2075 Chaitra (Back) [8]** |

**ANSWER**

**1. FCFS (First-Come First-Served / FIFO):**

Processes are executed in the order they arrive in the ready queue. Non-preemptive.

- Advantage: Simple to implement; no starvation (every process eventually runs).
- Disadvantage: Convoy Effect — one long process blocks all short processes behind it. Poor average waiting time for short processes. Not suitable for real-time systems.

**2. SJF (Shortest Job First) / SRTF (Shortest Remaining Time First):**

Select the process with the shortest CPU burst time next. SJF is non-preemptive; SRTF (preemptive SJF) preempts if a new shorter job arrives.

- Advantage: Minimizes average waiting time — provably optimal for minimizing AWT.
- Disadvantage: Requires knowing/estimating burst times in advance. Long processes may starve if short jobs keep arriving. Not used directly in RTOS (burst times unknown).

**3. Priority-Based Scheduling (Preemptive / Non-Preemptive):**

Each process assigned a priority. Preemptive: if new higher-priority task arrives, it immediately preempts current task. Non-preemptive: current task runs to completion before switching.

- Advantage (preemptive): High-priority tasks get immediate CPU — essential for RTOS.
- Disadvantage: Low-priority tasks may starve (never run). Solved by aging — gradually increasing priority of waiting tasks.
- Used in all RTOS: FreeRTOS, uC/OS, VxWorks all use preemptive priority scheduling.

**4. Round-Robin (RR):**

Each process is given a fixed time quantum (slice). After the slice, the process is moved to the back of the ready queue. Preemptive. Fair allocation.

- Advantage: Fair — no starvation. Good response time for interactive systems.

- Disadvantage: Performance depends on quantum size. Too small → high context switch overhead. Too large → degenerates to FCFS.
- Used in: GPOS (Linux CFS, Windows scheduler) and equal-priority task scheduling in RTOS.

**Q8. Three processes P1(CT=4ms,pri=1), P2(CT=10ms,pri=3), P3(CT=5ms,pri=2) enter together at t=0. P4(CT=3ms,pri=0) enters at t=5ms. Preemptive Priority (0=highest). Find WT, TAT, AWT, ATAT.**

**2081 Chaitra (Regular) [6]**

### ANSWER

```
Preemptive Priority (lower number = higher priority)
At t=0: P1(pri=1), P2(pri=3), P3(pri=2) ready. Highest = P1.

t=0 →4:  P1 runs (CT=4). Finishes at t=4.
t=4 →5:  Ready: P2(pri=3,rem=10), P3(pri=2,rem=5). Highest=P3. P3 starts.
t=5:     P4 arrives pri=0 (HIGHEST). PREEMPTS P3.
t=5 →8:  P4 runs (CT=3). Finishes at t=8.
t=8 →12: P3 resumes (rem=4ms). Finishes at t=12.
t=12→22: P2 runs (CT=10). Finishes at t=22.

Gantt: [P1:0-4][P3:4-5][P4:5-8][P3:8-12][P2:12-22]
```

| Process | Arrival | CT | Priority | Start | Finish | TAT | WT |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 4 | 1 | 0 | 4 | 4 | 0 |
| P2 | 0 | 10 | 3 | 12 | 22 | 22 | 12 |
| P3 | 0 | 5 | 2 | 4 | 12 | 12 | 7 |
| P4 | 5 | 3 | 0 | 5 | 8 | 3 | 0 |

**AWT = (0+12+7+0)/4 = 19/4 = 4.75 ms | ATAT = (4+22+12+3)/4 = 41/4 = 10.25 ms**

**Q9. Three processes P1(CT=10ms,pri=1), P2(CT=5ms,pri=3), P3(CT=7ms,pri=2) arrive at t=0. P4(CT=6ms,pri=0) enters at t=5ms. NON-Preemptive Priority. Find WT, TAT, AWT, ATAT.**

**2073 Bhadra (Regular) | 2076 Baisakh (Back) [8]**

### ANSWER

```
Non-Preemptive: once started, a task runs to completion.
At t=0: P1(pri=1), P2(pri=3), P3(pri=2) ready. Highest = P1. P1 starts.

t=0 →10: P1 runs (CT=10). P4 arrives at t=5 but CANNOT preempt P1.
t=10:    Ready: P4(pri=0), P3(pri=2), P2(pri=3). Highest = P4.
t=10→16: P4 runs (CT=6). Finishes.
t=16→23: P3 runs (CT=7). Finishes.
```

```
t=23→28: P2 runs (CT=5). Finishes.

Gantt: [P1:0-10][P4:10-16][P3:16-23][P2:23-28]
```

| Process | Arrival | CT | Priority | Start | Finish | TAT | WT |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 10 | 1 | 0 | 10 | 10 | 0 |
| P2 | 0 | 5 | 3 | 23 | 28 | 28 | 23 |
| P3 | 0 | 7 | 2 | 16 | 23 | 23 | 16 |
| P4 | 5 | 6 | 0 | 10 | 16 | 11 | 5 |

**AWT = (0+23+16+5)/4 = 44/4 = 11 ms | ATAT = (10+28+23+11)/4 = 72/4 = 18 ms**

**Q10. SJF Preemptive (SRTF): P1(CT=10), P2(CT=5), P3(CT=7) all arrive at t=0. P4(CT=2) arrives at t=2ms. Find WT, TAT, AWT, ATAT.**

**2081 Ashwin (Back) | 2079 Ashwin (Back) | 2077 Chaitra (Back) [4+2+6]**

### ANSWER

```
SRTF = Preemptive SJF: at every event, run the process with shortest
REMAINING time.

t=0: Ready: P1(rem=10), P2(rem=5), P3(rem=7). Shortest = P2(5). P2 starts.
t=2: P4 arrives (CT=2). Current: P2(rem=3), P4(rem=2). P4 shorter →
PREEMPT P2.
t=2→4:   P4 runs (CT=2). P4 finishes. TAT=4-2=2, WT=2-2=0.
t=4→7:   P2 resumes (rem=3). P2 finishes. TAT=7-0=7, WT=7-5=2.
t=7→14:  P3 runs (rem=7). P3 finishes. TAT=14-0=14, WT=14-7=7.
t=14→24: P1 runs (rem=10). P1 finishes. TAT=24-0=24, WT=24-10=14.

Gantt: [P2:0-2][P4:2-4][P2:4-7][P3:7-14][P1:14-24]
```

| Process | Arrival | CT | Priority | Start | Finish | TAT | WT |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 10 | — | 14 | 24 | 24 | 14 |
| P2 | 0 | 5 | — | 0 | 7 | 7 | 2 |
| P3 | 0 | 7 | — | 7 | 14 | 14 | 7 |
| P4 | 2 | 2 | — | 2 | 4 | 2 | 0 |

**AWT = (14+2+7+0)/4 = 23/4 = 5.75 ms | ATAT = (24+7+14+2)/4 = 47/4 = 11.75 ms**

**Q11. Round-Robin Scheduling (quantum=20ms): P1=53ms, P2=17ms, P3=68ms, P4=24ms, all arrive at t=0. Find WT, TAT, AWT, ATAT.**

**2082 Kartik (Back) | 2078 Chaitra (Back) | 2080 Chaitra (Regular) [6]**

## ANSWER

```
Round-Robin (Q=20ms): cycle through ready queue; each gets 20ms max.

t=0  →20:  P1 runs 20ms.  P1 rem=33
t=20 →37:  P2 runs 17ms.  P2 DONE. Finish=37, TAT=37, WT=37-17=20
t=37 →57:  P3 runs 20ms.  P3 rem=48
t=57 →77:  P4 runs 20ms.  P4 rem=4
t=77 →97:  P1 runs 20ms.  P1 rem=13
t=97 →117: P3 runs 20ms.  P3 rem=28
t=117→121: P4 runs 4ms.   P4 DONE. Finish=121, TAT=121, WT=121-24=97
t=121→134: P1 runs 13ms.  P1 DONE. Finish=134, TAT=134, WT=134-53=81
t=134→162: P3 runs 28ms.  P3 DONE. Finish=162, TAT=162, WT=162-68=94

Gantt: [P1:0-20][P2:20-37][P3:37-57][P4:57-77][P1:77-97][P3:97-117]
       [P4:117-121][P1:121-134][P3:134-162]
```

| Process | CT (ms) | Finish (ms) | TAT (ms) | WT (ms) |
|---|---|---|---|---|
| P1 | 53 | 134 | 134 | 81 |
| P2 | 17 | 37 | 37 | 20 |
| P3 | 68 | 162 | 162 | 94 |
| P4 | 24 | 121 | 121 | 97 |

**AWT = (81+20+94+97)/4 = 292/4 = 73 ms | ATAT = (134+37+162+121)/4 = 454/4 = 113.5 ms**

**Q12. FCFS Scheduling: P1(AT=0,CT=8), P2(AT=1,CT=4), P3(AT=2,CT=9), P4(AT=3,CT=5). Find WT, TAT, AWT, ATAT.**

**Model Question | 2074 Ashwin (Back) [6]**

## ANSWER

```
FCFS: Non-preemptive. Execute in order of arrival.

t=0 →8:  P1 runs (CT=8). P1 Finish=8, TAT=8-0=8, WT=0-0=0
t=8 →12: P2 runs (CT=4). P2 Finish=12, TAT=12-1=11, WT=8-1=7
t=12→21: P3 runs (CT=9). P3 Finish=21, TAT=21-2=19, WT=12-2=10
t=21→26: P4 runs (CT=5). P4 Finish=26, TAT=26-3=23, WT=21-3=18

Gantt: [P1:0-8][P2:8-12][P3:12-21][P4:21-26]
```

| Process | Arrival | CT | Priority | Start | Finish | TAT | WT |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 8 | — | 0 | 8 | 8 | 0 |
| P2 | 1 | 4 | — | 8 | 12 | 11 | 7 |
| P3 | 2 | 9 | — | 12 | 21 | 19 | 10 |
| P4 | 3 | 5 | — | 21 | 26 | 23 | 18 |

**AWT = (0+7+10+18)/4 = 35/4 = 8.75 ms | ATAT = (8+11+19+23)/4 = 61/4 = 15.25 ms**

**Q13. SJF Non-Preemptive: P1(AT=0,CT=7), P2(AT=2,CT=4), P3(AT=4,CT=1), P4(AT=5,CT=4). Find WT, TAT, AWT, ATAT.**

**Model Question | 2077 Jestha (Back) [6]**

### ANSWER

```
SJF Non-Preemptive: when CPU becomes free, pick shortest job from ready
queue.

t=0: Only P1 available. P1 starts (CT=7).
t=7: P1 finishes. Ready: P2(CT=4), P3(CT=1), P4(CT=4) [all arrived by
t=7].
     Shortest = P3(CT=1). P3 starts.
t=8: P3 finishes. Ready: P2(CT=4), P4(CT=4). Tie → pick P2 (arrived
first).
t=8→12: P2 runs. Finishes.
t=12→16: P4 runs. Finishes.

Gantt: [P1:0-7][P3:7-8][P2:8-12][P4:12-16]
```

| Process | Arrival | CT | Priority | Start | Finish | TAT | WT |
|---|---|---|---|---|---|---|---|
| P1 | 0 | 7 | — | 0 | 7 | 7 | 0 |
| P2 | 2 | 4 | — | 8 | 12 | 10 | 6 |
| P3 | 4 | 1 | — | 7 | 8 | 4 | 3 |
| P4 | 5 | 4 | — | 12 | 16 | 11 | 7 |

**AWT = (0+6+3+7)/4 = 16/4 = 4 ms | ATAT = (7+10+4+11)/4 = 32/4 = 8 ms**

**Q14. Round-Robin (quantum=5ms): P1(AT=0,CT=12), P2(AT=0,CT=6), P3(AT=0,CT=10), P4(AT=0,CT=4). Find WT, TAT, AWT, ATAT.**

**Model Question | 2075 Bhadra (Back) [6]**

### ANSWER

```
Round-Robin (Q=5ms): all arrive at t=0, ready queue order: P1,P2,P3,P4

t=0  →5:  P1 runs 5ms.  P1 rem=7
t=5  →10: P2 runs 5ms.  P2 rem=1
t=10 →15: P3 runs 5ms.  P3 rem=5
t=15 →19: P4 runs 4ms.  P4 DONE. Finish=19, TAT=19, WT=19-4=15
t=19 →24: P1 runs 5ms.  P1 rem=2
t=24 →25: P2 runs 1ms.  P2 DONE. Finish=25, TAT=25, WT=25-6=19
t=25 →30: P3 runs 5ms.  P3 DONE. Finish=30, TAT=30, WT=30-10=20
t=30 →32: P1 runs 2ms.  P1 DONE. Finish=32, TAT=32, WT=32-12=20

Gantt: [P1:0-5][P2:5-10][P3:10-15][P4:15-19][P1:19-24][P2:24-25]
       [P3:25-30][P1:30-32]
```

| Process | CT (ms) | Finish (ms) | TAT (ms) | WT (ms) |
|---|---|---|---|---|
| P1 | 12 | 32 | 32 | 20 |
| P2 | 6 | 25 | 25 | 19 |
| P3 | 10 | 30 | 30 | 20 |
| P4 | 4 | 19 | 19 | 15 |

**AWT = (20+19+20+15)/4 = 74/4 = 18.5 ms | ATAT = (32+25+30+19)/4 = 106/4 = 26.5 ms**

| **Q15. What is a semaphore? Explain producer-consumer problem and how semaphores solve it. Explain message queues in RTOS.** | **2075 Bhadra (Back) \| 2072 Magh (Back) [4+4]** |
|---|---|

### ANSWER

**Semaphore: (See Q5 for full definition.)**

**Producer-Consumer Problem:**

The producer-consumer (bounded buffer) problem involves two tasks sharing a fixed-size buffer: a producer task writes data into the buffer, and a consumer task reads from it. The challenges are:

- Producer must not write when buffer is FULL (overflow).
- Consumer must not read when buffer is EMPTY (underflow).
- Both must not access the buffer simultaneously (race condition).

**Solution using Semaphores:**

```
// Three semaphores/mutex:
Semaphore empty = N;   // counts empty slots (init = buffer size N)
Semaphore full  = 0;   // counts full slots (init = 0)
Mutex   mutex   = 1;   // protects buffer access (binary semaphore)

// PRODUCER task:                    // CONSUMER task:
while(1) {                            while(1) {
  produce_item(item);                   sem_wait(full);     // wait for item
  sem_wait(empty);  // wait for slot    sem_wait(mutex);    // lock buffer
  sem_wait(mutex);  // lock buffer      item = remove_from_buffer();
  add_to_buffer(item);                 sem_signal(mutex); // unlock
  sem_signal(mutex); // unlock          sem_signal(empty); // signal slot
free
  sem_signal(full);  // signal item     consume_item(item);
}                                     }
```

### Message Queues in RTOS:

A message queue is an RTOS kernel object that allows tasks to communicate by sending and receiving fixed-size messages (data structures) in a FIFO order. It is the primary inter-task communication (ITC) mechanism in RTOS.

- Thread-safe: The kernel protects the queue from simultaneous access.
- Blocking: A task trying to read an empty queue blocks (sleeps) until a message arrives. A task trying to write a full queue blocks until space is available.
- Timeout: Tasks can wait with a timeout (osMessageQueueGet with osWaitForever or timeout_ms).

```
// Create queue: 10 messages, each 4 bytes
osMessageQueueId_t queue = osMessageQueueNew(10, sizeof(uint32_t), NULL);

// Producer task:
uint32_t data = sensor_read();
osMessageQueuePut(queue, &data, 0, osWaitForever);  // blocks if full

// Consumer task:
uint32_t rxData;
osMessageQueueGet(queue, &rxData, NULL, osWaitForever);  // blocks if
empty
process(rxData);
```

| **Q16. Explain device drivers in the context of RTOS. What are the different types of device drivers?** | **2080 Ashwin (Back) \| 2078 Poush (Back) [4+4]** |
|---|---|

**ANSWER**

### Device Drivers in RTOS:

A device driver is a software module that provides a standardized interface between the RTOS kernel / application and a hardware peripheral (UART, SPI, ADC, GPIO, CAN, USB, etc.). It abstracts hardware-specific details — the application calls generic read/write/ioctl functions without needing to know register-level details.

```
Application Layer:  read(uart, buf, len)  write(uart, buf, len)
                                ↕
Device Driver API:  uart_init(), uart_read(), uart_write(), uart_ioctl()
                                ↕
Hardware Register:  USART->DR, USART->SR, USART->BRR  (STM32 UART)
```

**Key Responsibilities of a Device Driver:**

- Initialization: Configure hardware registers (baud rate, mode, interrupts enabled).
- Data Transfer: Implement read/write for the peripheral — polling, interrupt-driven, or DMA-based.
- Interrupt Handling: Register and handle the peripheral's ISR; signal RTOS task via semaphore or message queue.
- Error Handling: Detect and report hardware errors (overrun, parity error, framing error).
- Power Management: Enable/disable peripheral clocks; implement low-power modes.

**Types of Device Drivers:**

**1. Polled (Programmed I/O) Driver:** Driver continuously polls the hardware status register in a loop until data is ready. Simple but wastes CPU. Used only for very fast or simple peripherals.

**2. Interrupt-Driven Driver:** Hardware generates interrupt when ready; ISR processes data and signals the waiting task via semaphore. CPU is free between transfers. Most common in RTOS.

**3. DMA-Driven Driver:** DMA controller transfers entire blocks of data without CPU involvement. Driver configures DMAC; DMA completion triggers interrupt. Used for high-speed bulk transfers (UART DMA, SPI DMA, ADC continuous mode).

**4. Character Device Driver:** Handles devices that produce/consume a stream of bytes (UART, terminal). Typically uses ring buffers + interrupts.

**5. Block Device Driver:** Handles devices accessed in fixed-size blocks (SD card, Flash). Includes wear leveling, caching.

**6. Virtual Device Driver:** Software abstraction with no physical hardware — e.g., a software timer driver, a loopback UART driver for testing.

# Summary: Topics Covered in Chapter 6

| S.N. | Topic | Exam(s) Asked | Marks |
|---|---|---|---|
| 1 | RTOS vs GPOS — full comparison table | 2081 Chaitra, 2079 Chaitra, 2078 Baishakh (annual) | 3–4 |
| 2 | Process vs Thread — comparison table | 2081 Chaitra, 2079 Chaitra (annual) | 3 |
| 3 | Kernel: Monolithic vs Microkernel | 2073 Bhadra, 2076 Baisakh | 3+3 |
| 4 | RT Kernel Major Functions (8 functions) | 2081 Ashwin | 4 |
| 5 | Process Life Cycle (5 states + transitions) | 2081 Ashwin, 2077 Magh | 4 |
| 6 | Task Control Block (TCB) — structure | 2081 Ashwin, 2077 Magh | 2 |
| 7 | Context Switching — situations + steps | 2080 Chaitra, 2074 Bhadra | 2+6 |
| 8 | Task Synchronization: Semaphore + Mutex | 2075 Bhadra, 2072 Magh, 2076 Chaitra | 2+3+3 |
| 9 | Priority Inversion + Priority Inheritance | 2075 Bhadra, 2072 Magh | 4 |
| 10 | Deadlock: Definition + Coffman Conditions | 2082 Kartik, 2073 Bhadra, 2078 Chaitra | 6 |
| 11 | Deadlock Handling Techniques (4 methods) | 2082 Kartik, 2073 Bhadra | 6 |
| 12 | Scheduling Algorithms Theory (FCFS/SJF/Pri/RR) | 2080 Ashwin, 2079 Jestha, 2075 Chaitra | 8 |
| 13 | Preemptive Priority Scheduling — Numerical | 2081 Chaitra, 2079 Chaitra, 2073 Bhadra (annual) | 6 |
| 14 | Non-Preemptive Priority — Numerical | 2073 Bhadra, 2076 Baisakh | 8 |
| 15 | SJF Preemptive (SRTF) — Numerical | 2081 Ashwin, 2079 Ashwin, 2077 Chaitra (annual) | 6 |
| 16 | Round-Robin — Numerical (Q=20ms) | 2082 Kartik, 2078 Chaitra, 2080 Chaitra (annual) | 6 |
| 17 | Round-Robin — Numerical (Q=5ms) | 2075 Bhadra, Model Q | 6 |
| 18 | FCFS — Numerical | 2074 Ashwin, Model Q | 6 |
| 19 | SJF Non-Preemptive — Numerical | 2077 Jestha, Model Q | 6 |
| 20 | Producer-Consumer Problem + Semaphores | 2075 Bhadra, 2072 Magh | 4+4 |
| 21 | Message Queues in RTOS | 2075 Bhadra, 2072 Magh | 4 |
| 22 | Device Drivers — Types + RTOS context | 2080 Ashwin, 2078 Poush | 4+4 |